

# Engenharia de Prompts · Usando Condições

```
from dotenv import load_dotenv
from groq import Groq
import os
load_dotenv()

client = Groq(
    api_key=os.environ.get("GROQ_API"), # This is the default and can be omitted
)
```

## Usando Condicionais Básicos

```
system_prompt = """
Você é um nutricionista virtual adaptativo. Siga estas condições:

SE a pergunta for BÁSICA:
- Explique de forma simples
- Use exemplos do dia a dia
- Evite termos técnicos

SE a pergunta for AVANÇADA:
- Use linguagem científica
- Cite termos nutricionais
- Sugira leituras adicionais

SE a pergunta for AMBÍGUA:
- Peça esclarecimentos
- Ofereça opções de interpretação
"""

user_prompt = "O que é proteína?"

response = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt}
    ],
    temperature=0.7
)

print("=== Exemplo 1: Básico ===")
print(response.choices[0].message.content)
```

## Adaptação ao Nível do Usuário

```
system_prompt = """
Você é um coach de saúde que adapta explicações conforme o nível:

INICIANTE (palavras: "novo", "começando", "não sei nada"):
- Explicações muito simples
- Exemplos cotidianos
- Passo a passo claro

INTERMEDIÁRIO (palavras: "já sei", "tenho alguma experiência"):
- Respostas diretas
- Pequenos detalhes técnicos
- Sugestões práticas

AVANÇADO (palavras: "experiente", "profissional", "avançado"):
```

```
- Respostas técnicas
- Citações científicas
- Estratégias avançadas
"""
```

```
user_prompt_iniciante = "Sou novo em dieta, como começo a comer melhor?"
```

```
user_prompt_avancado = "Sou nutricionista, explique estratégias de periodização alimentar."
```

```
resp_iniciante = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt_iniciante}
    ],
    temperature=0.6
)
```

```
resp_avancado = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt_avancado}
    ],
    temperature=0.6
)
```

```
print("\n=== Exemplo 2: Iniciante ===")
print(resp_iniciante.choices[0].message.content)
```

```
print("\n=== Exemplo 2: Avançado ===")
print(resp_avancado.choices[0].message.content)
```

## Usando Múltiplas Condições

```
system_prompt = """
Você é um orientador de dieta. Adapte recomendações com base nas condições:
```

```
OBJETIVO EMAGRECIMENTO:
```

- Foque em déficit calórico
- Sugira exercícios leves
- Indique alimentos pouco calóricos

```
OBJETIVO GANHO DE MASSA:
```

- Foque em superávit calórico
- Sugira treino de força
- Recomende proteínas

```
VALOR BAIXO (< R$ 500/mês):
```

- Sugira alimentos acessíveis
- Foque em custo-benefício

```
VALOR ALTO (> R$ 2000/mês):
```

- Sugira alimentos premium
  - Inclua suplementos importados
- ```
"""
```

```
user_prompt = "Tenho R$ 400 por mês e quero emagrecer comendo bem."
```

```
resp_multi = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[
        {"role": "system", "content": system_prompt},
```

```

        {"role": "user", "content": user_prompt}
    ],
    temperature=0.6
)

print("\n=== Exemplo 3: Múltiplas Condições ===")
print(resp_multi.choices[0].message.content)

```

## Usando Fallback

```

system_prompt = """
Você é um chatbot de nutrição. Regras:

SE a pergunta for sobre RECEITAS:
- Sugira 3 opções rápidas

SE for sobre EXERCÍCIOS:
- Recomende treinos simples

SE for sobre SUPLEMENTOS:
- Explique benefícios e riscos

CASO CONTRÁRIO (fallback):
- Peça mais detalhes
- Sugira opções de tema
- Sempre responda com simpatia
"""

user_prompt = "Vocês podem me ajudar a correr mais rápido na lua?"

```

```

resp_fallback = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt}
    ],
    temperature=0.7
)

print("\n=== Exemplo 5: Fallback ===")
print(resp_fallback.choices[0].message.content)

```